

GUIA DE DEMOSTRACION EN VIVO

TRADING ROI EXPLAINER & TRANSACTION SIMULATOR API (GEMINI 3.5)

Esta guía contiene la secuencia detallada de comandos paso a paso para realizar una demostración en vivo impecable y espectacular de tu trabajo final de integración (Marcelo Gilardoni).

Paso 0: Preparación Inicial de la Consola

Abre una terminal de PowerShell en tu computadora y ejecuta los siguientes comandos:

1. Posicionarte en la carpeta de tu trabajo final:
`cd C:\OpenCode\1-integracion-tbf`
2. Activar el entorno virtual de Python:
`.\venv\Scripts\Activate.ps1`

Paso 1: Demostración de Gemini 3.5 en Consola (Streaming vs No Streaming)

Compara la respuesta síncrona contra la visualización en vivo de tokens.

1. Ejecutar CON STREAMING:
`python codigo/scripts/demo_gemini_streaming.py --stream`
 - Resalta la velocidad percibida instantánea (<1s).
 - Resalta la caja de comprobación de enmascaramiento: 'Maria Gomez' -> 'M**** G****'.
2. Ejecutar SIN STREAMING:
`python codigo/scripts/demo_gemini_streaming.py`
 - Muestra la espera síncrona estática (20s) para justificar el uso de streaming en producción.

Paso 2: Levantar el Servidor Web e Interfaz Swagger (FastAPI)

Muestra el endpoint cerrado con llave ante accesos no autorizados:

1. Inicia el servidor FastAPI:
`python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000 --app-dir "codigo"`
2. Abre la documentación interactiva en el navegador:
`http://127.0.0.1:8000/docs`
3. Demuestra la protección X-API-Key: Sin cabecera el servidor bloquea con error HTTP 401. Al ingresar la clave 'trading-secret-key-123' en la autorización superior, retorna 200 OK.

Paso 3: Demostración de Observabilidad con OpenTelemetry (OTel)

Demuestra el sistema de diagnóstico de trazas y logs en tiempo real:

1. Desglose de Spans: Muestra los logs '[OTel Span]' con las latencias por capa (conector, adaptador, Gemini).
2. Correlación de Logs: Resalta que cada registro de log posee el [TraceId: ... | SpanId: ...].
3. Diagnóstico de Errores: Ejecuta un curl enviando un activo no válido como 'XYZ' y observa el Span de ERROR.
4. Simulación de Falla en Base de Datos: Cambia 'mock_db.json' a 'Solo Lectura' en Windows. Verás cómo el sistema aborta de inmediato la llamada a Gemini para resguardar la consistencia financiera y le indica al cliente que vuelva a intentar (Retornando error 503).

Paso 4: Observabilidad en la Nube con Langfuse (LLM Ops)

1. Abre en tu navegador la consola web de Langfuse Cloud:

<https://us.cloud.langfuse.com>

2. Monitoreo de Trazas: Señala el costo en USD, el conteo de tokens de Gemini 3.5, el enmascaramiento de datos de 'Maria Gomez' a 'M**** G****' en las trazas y prompts.
3. Visualización de Dashboards Consolidados (Gráficos Interactivos):
 - Ejecuta el poblador de gráficos en PowerShell:
python codigo/scripts/generate_dashboard_data.py
 - Abre la pestaña 'Dashboards' en tu cuenta de Langfuse Cloud para enseñar las métricas consolidadas reales de costos totales acumulados en dólares, volumen total de transacciones y promedios de latencia.

Paso 5: Evaluacion de Calidad con Datasets en Langfuse

1. Crear el Dataset 'trading_agent_evaluation' en la nube:
python codigo/scripts/create_langfuse_dataset.py
2. Correr la evaluacion en vivo de los 20 casos de prueba con Gemini:
python codigo/scripts/run_langfuse_evaluation.py
3. En la consola de Langfuse, muestra la pestaña 'Runs' con la comparativa lado a lado.
4. ¿Cómo funciona el Juez Cognitivo de DeepEval (Juez Gemini)?
 - Opera bajo el estándar 'LLM-as-a-Judge': Toma la Entrada, la Respuesta real de tu agente y el Resultado Esperado de Referencia.
 - Envía los datos a tu Juez Gemini (DeepEvalGeminiJudge), el cual audita el texto, valida la consistencia matemática, el enmascaramiento de PII y la coherencia del Stop-Loss, otorgando un Score de 0.0 a 1.0 y detallando su razonamiento lógico.

Paso 6: Pruebas y Medicion de Latencias (p50 y p95)

1. Ejecutar las pruebas unitarias y de enmascaramiento de datos:
python -m pytest codigo/tests/test_trading_api.py -v
2. Ejecutar el benchmark para medir p50 y p95 latencia (10 llamadas consecutivas):
python codigo/scripts/benchmark_latencia.py
Se generara la tabla de percentiles en pantalla y se guardara en benchmark_results.json.